Learning to dock a pallet truck from one camera: imitating a privileged expert with a vision-language-action policy in simulation

Agile AMR project · mini research report · status as of 2026-10-10 · all results are from simulation

Abstract

Docking a pallet truck's tines into a pallet needs about 1 cm lateral and 1–3° heading accuracy at low speed. We study whether a small vision-language-action model (SmolVLA, 450 M parameters) can learn this end to end from a single forward-facing RGB camera, the steering angle and two backrest-switch bits, by imitating a scripted expert that reads the true poses in Isaac Sim. Over seven policy generations, the share of 128 fixed random starts docked within a 60 s budget rose from **41 % to 94 %** (95 % interval 88–97 %), with no jams, a median docking time of 19 s (the expert needs 18 s) and a 90th percentile of 24 s. Two diagnoses, both found by analysing closed-loop failures rather than training loss, produced most of the gain: hiding the truck's own velocity from the policy removed a copycat shortcut ("stationary, therefore stay stationary"; 41 → 66 %), and redesigning the expert so that its stop-and-fix manoeuvres were rare, decisive and learnable from the image (66 → 94 %). The results are simulation-only, from single training runs, on starts drawn from the training distribution; robustness outside it and the sim-to-real gap are untested. We lay out a gated path to hardware: stress tests and randomisation in simulation, latency-aware onboard inference, shadow-mode recording, supervised low-speed trials behind an independent safety layer, and fine-tuning with real data.

1. Introduction

A pallet truck must slide two tines into the pockets of a pallet whose pose varies from approach to approach. In the simulation used here the pockets tolerate roughly −6…+4 cm of lateral error and under 3° of heading error, so the last metre of the approach is a precision task, performed in reverse (tines first) by a tricycle-steered vehicle whose steered wheel trails behind it.

The classical solution estimates the pallet pose with a perception module and tracks it with a planner and controller. We instead ask whether **one learned policy, from camera pixels to steering and wheel-speed commands**, can do the whole job. We chose a vision-language-action (VLA) model for three reasons: it is a single network that can later take language instructions (lifting, aisle manoeuvres); it needs no explicit pose estimator at run time; and it starts from pretrained visual features. We train it by **imitation of a privileged scripted expert**: in simulation the expert reads the true truck and pallet poses, so labels are free, while the policy sees only what a real camera and encoders would provide.

Contributions. (1) A complete simulation pipeline: scene, a state-machine expert with the truck's full motion set, data collection, training, closed-loop evaluation and viewers. (2) A policy that docks

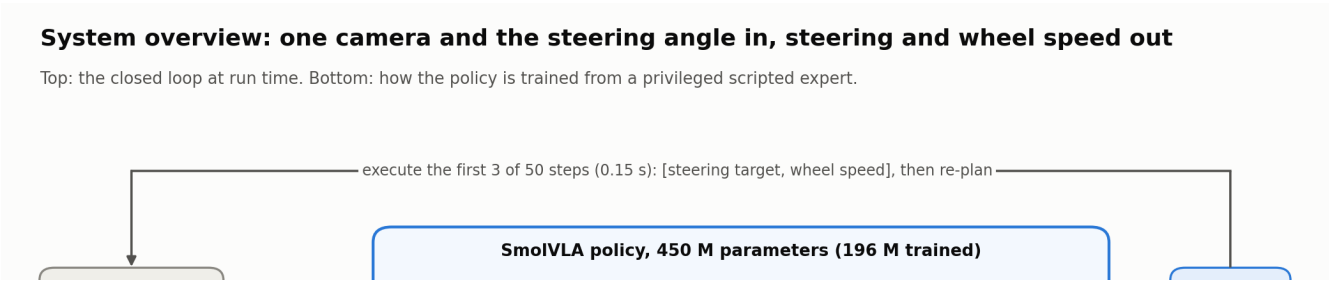
94 % of fixed random starts in 60 s. (3) Two measured failure mechanisms, a copycat effect from the velocity inputs and an expert whose stop-and-fix decisions the camera cannot resolve, with the interventions that removed them. (4) An evaluation protocol (fixed starts, intervals, paired tests) that exposed how misleading single 64-episode evaluations of a stochastic policy are. (5) A staged plan for hardware deployment (Section 6).

2. Task and platform

Item	Definition
Goal	Insert the tines until the carriage backrest reaches the pallet's docking face, where the real truck's two switch blocks sit. No lift.
Scene	Open floor. Plastic pallet (1200 × 1000 × 160 mm) docked through its 1000 mm face; the pallet is a fixed (kinematic) body, so a jam shows as a stalled truck.
Starts	Pallet yaw uniform over ±180°. Truck 3–4 m from the face (tine tips), lateral offset ±0.3 m, heading offset ±15°.
Instruction	Frozen text prompt "dock into the pallet" (the language path is not trained).
Success	Backrest within 5 mm of the face and not more than 3 cm past it when the episode ends, within the time budget.
Budget and stop rules	60 s (evaluation protocol); an episode also ends if the policy commands motion for 2 s without the truck moving (jam). The expert needs a median 18 s, p95 38 s.
Robot	Tricycle pallet truck: steered traction wheel at the chassis end, fixed load-wheel axle near the tine tips; wheelbase 1.13 m; steering limited to −85°…+155° with a rate limit of 0.6 rad/s (a 90° swing takes 2.6 s). At 90° steering the truck pivots in place about the load-wheel axle.
Sensing	Fork-carriage camera (Orbbec Femto Bolt, 256×144 RGB after scaling), steering-angle encoder, two backrest switches. Depth is rendered but unused.
Simulator	Isaac Sim 6.0.1 / Isaac Lab, PhysX at 60 Hz, control and camera at 20 Hz; environments spaced 40 m apart so that neighbours are never visible.

With u the traction-wheel ground speed and δ the steering angle, the body forward speed is $v_x = u \cos \delta$ and the yaw rate $\omega = u \sin \delta / L$. A path-following steer is $\delta = \operatorname{atan}(2 L y_b / d^2)$ for a look-ahead point at lateral offset y_b and distance d, valid in both driving directions.

3. Method



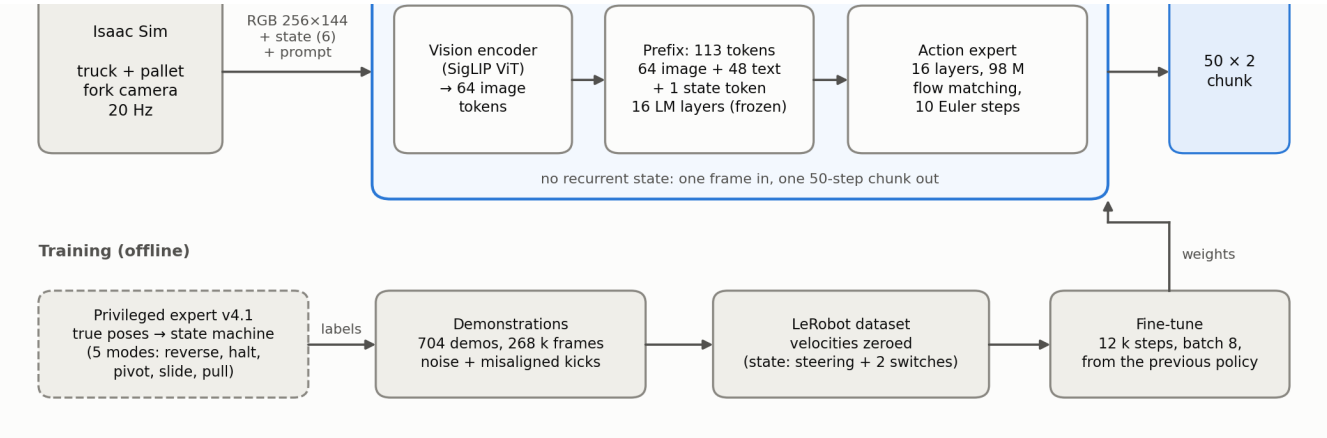


Figure 1. Top: the closed loop at run time. Bottom: how the policy is trained from the privileged expert.

3.1 The privileged expert

The expert outputs a = [steering target δ, *wheel ground speed u*] each control step from the true poses. Version 4.1 is a five-mode state machine that always chooses the smallest move that works:

- **reverse:** tines-first pure pursuit of the pallet centre-line; 0.6 m/s when lined up, 0.4 m/s while correcting, 0.3 m/s near the face; brakes to stop with the backrest at the face.
- **halt:** brake to standstill, then choose a fix. Heading error up to 4°, outside the pocket: **slide** (forward micro-arc with the steered wheel leading). Larger heading error: **pivot** in place at 90° steering. Lateral error: pivot to a tilt, slide along it, pivot back. Wedged or off inside the pocket: **pull** straight out only far enough, then fix.

The expert evolved over four generations (v1: reverse pursuit only; an abandoned 3 m retreat variant, rejected as too long for a warehouse; v3: standoff with pivot and slide; v4: the full set above with real speeds). **v4.1** changed three things in response to the analysis in Section 4.2:

Parameter	**v4**	**v4.1**
"Aligned enough to enter" band	2 cm / 2°	**3.5 cm / 2.5°**
Committed-drift limit (stop if exceeded inside)	3.5 cm / 2.5°	4 cm / 3° (the pocket tolerance)
Pivot yaw rate (minimum / maximum)	0.08 / 0.35 rad/s	**0.18 / 0.5 rad/s** (at least 0.2 m/s at the wheel)
Pivot stop tolerance	0.5°	1.5°
Perturbation kicks	drive in until wedged	**only while the expert is in reverse; end when it brakes**

3.2 Demonstrations

Demonstrations are collected headless, 32 environments in parallel. Each step records the camera image, the measured state, the expert's label, the action actually executed, privileged poses and the expert mode. To put recoveries into the data we perturb the *executed* action while keeping the

clean expert action as the label (noise injection in the style of DART): an Ornstein–Uhlenbeck perturbation on half of the environments (σ = 0.08 rad steering, 0.03 m/s speed, 0.5 s correlation, tapering to zero within 1 m of the face), and misaligned "kicks" that steer the truck off-line until the expert objects.

The v4.1 set has 704 demonstrations (268,373 frames, 100 % docked, median 18 s, p90 22 s). 128 of them (18 %) contain a fix: 110 kicked demonstrations and 18 of the 594 others (3.0 %, caused by the approach noise). The v4 set had 18 % fixes among its *normal* demonstrations alone.

3.3 The policy

SmolVLA is a small vision-language model (SmolVLM2-500M, truncated to 16 layers) plus a 98 M-parameter **action expert** that generates actions by flow matching.

Block	Parameters	Training
Vision encoder (ViT, 12 layers, patch 16, 512×512)	86.4 M	trained from v3 on
Connector (pixel shuffle ×4)	11.8 M	trained from v3 on
Language model (16 layers, width 960) and embeddings	252 M	frozen
Action expert (16 layers, width 720)	98.3 M	trained
Input/output projections	1.6 M	trained

Inputs. The image (1 camera, 256×144) is padded to 512×512 and becomes 64 visual tokens; the fixed prompt becomes 48 tokens; the state becomes one token. For the final policies the state is the steering angle and the two switch bits, with the wheel speed, forward speed and yaw rate **zeroed** (Section 4.2). The 113-token prefix passes through the language layers once per observation.
Output. The action expert denoises a 50-step chunk of [δ, a]; the first 3 steps (0.15 s) are executed, then the policy re-plans. There is no recurrent state: the policy is a function of the current frame and state.

Flow matching. Training draws noise $\varepsilon \sim N(0, I)$ and a time t, forms $x_t = t\cdot\varepsilon + (1 - t)\cdot a$ from a real expert chunk a, and regresses the velocity $\varepsilon - a$ with a mean-squared loss. At run time the chunk is produced by Euler integration, $x \leftarrow x + \Delta t \cdot v_\theta(x, t, \text{obs})$, ten steps from $t = 1$ to 0. Because the generation starts from random noise, **the policy is stochastic**: the same start can end differently on two rolls.

3.4 Training and evaluation protocol

Each generation fine-tunes the previous one (AdamW, peak learning rate $2–3\cdot10^{-5}$ with cosine decay, batch 8 on an 8 GB GPU, 12 k–28 k steps, no image augmentation; 1.5–4 h per run). The headline evaluation uses **128 fixed starts** (8 batches of 16, seeds fixed so every policy sees the same starts), a 60 s budget, and reports the share docked with a 95 % Wilson interval, jams, time to dock, and a paired sign test between policies on identical starts. Earlier generations were evaluated with one roll of 32 or 64 starts and 60–80 s caps; those numbers are kept for history but are not

comparable.

4. Results

4.1 Main result

	v4	v4nv	v4.1nv
Docked within 60 s	53/128 = 41 % (CI 33–50)	85/128 = 66 % (CI 58–74)	**120/128 = 94 % (CI 88–97)**
Jams	3	1	**0**
Median / p90 time to dock	18 s / 48 s	20 s / 46 s	**19 s / 24 s**
Docked by 20 s / 30 s / 45 s	32 / 32 / 34 %	39 / 48 / 59 %	**73 / 88 / 91 %**
Starts that pivot at all	–	63 of 128	**7 of 128**
Never entered the pallet	12	39	**8**

("nv" = no velocity inputs; v4.1nv additionally uses the v4.1 expert's data.) Paired on the same starts, v4nv vs v4: 45 docked only with v4nv, 13 only with v4 ($p < 0.001$); v4.1nv vs v4nv: 38 only with v4.1nv, 3 only with v4nv ($p < 0.001$).

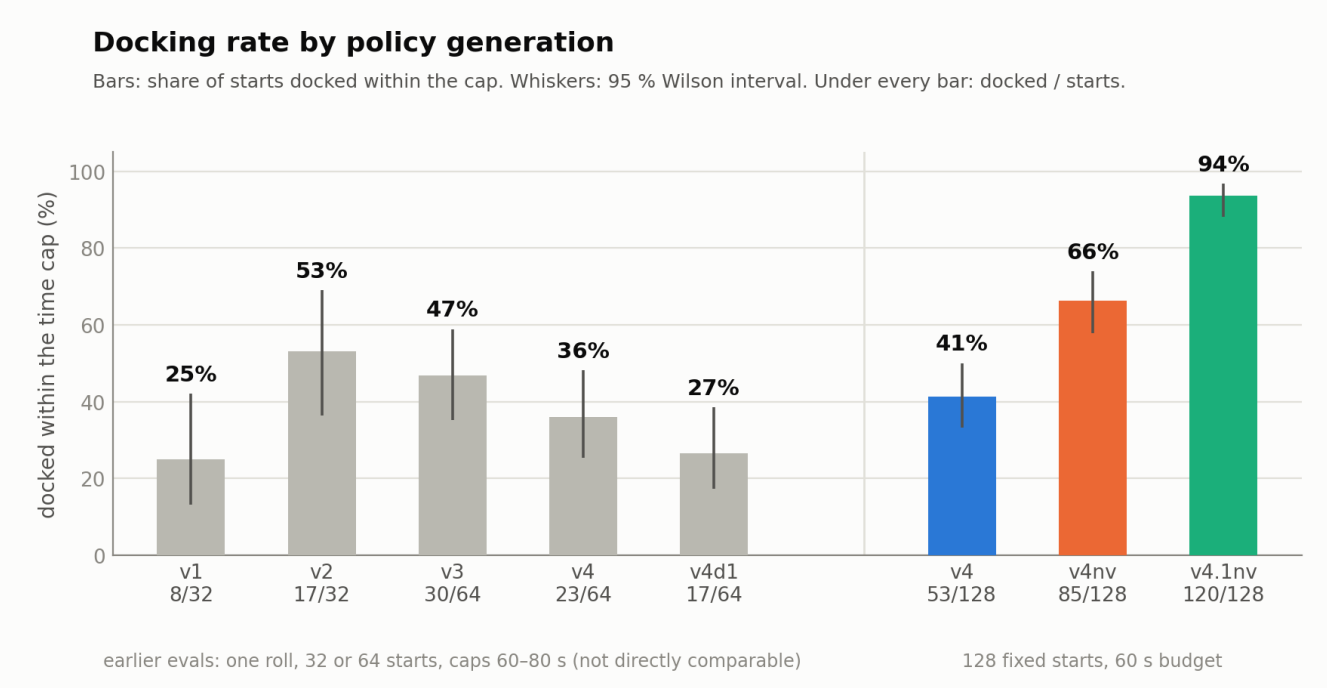
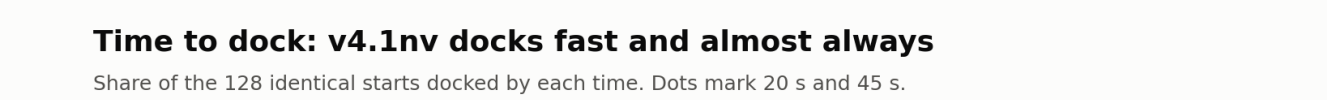


Figure 2. Docking rate by policy generation. The five grey bars are single rolls under earlier protocols (not directly comparable, wide intervals); the three coloured bars share the 128-start protocol.



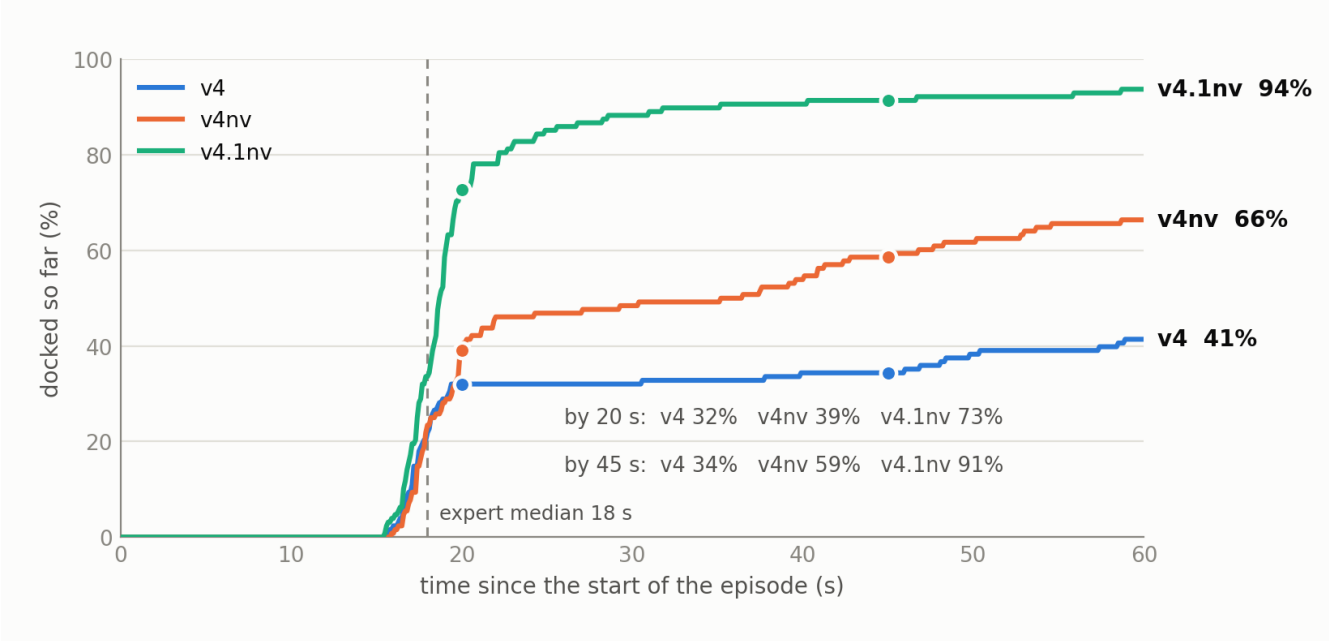


Figure 3. Share of the 128 identical starts docked by each time. v4 plateaus after about 20 s, v4nv keeps docking, v4.1nv docks fast and almost always.

4.2 Where the gains came from

Step 1, hiding the velocity (41 % → 66 %). v4 spent a median 26 s sitting still outside the pallet after entering it. In the training data 21 % of frames have the truck stationary and 94 % of those are labelled "speed ≈ 0" (waiting for the steering to swing, braking, settled). A policy that can read its own velocity learns the near-perfect rule *stationary ⇒ output zero*, so once its own mistake leaves it stationary it stays there: the copycat (causal-confusion) effect [5]. Zeroing wheel speed, forward speed and yaw rate in the state, in training and at run time, raised docking from 41 % to 66 % and cut "entered but did not finish" from 63 to 4 episodes.

Step 2, a learnable expert (66 % → 94 %). The remaining failures of v4nv were almost all the same: 39 of 43 never entered, and 37 of those started a pivot at the standoff and sat at 90° steering for a median 38 s while the heading error was only 1.4°. Three measurements explain it:

1. At 90° steering the truck's rotation depends gradually on the commanded wheel speed (table below); the stalled frames mostly sit in the weak zone.
2. In 53 % of the pivot frames the heading was already within 2°, where the expert itself would not pivot.
3. The expert's alignment band (2 cm / 2°) was tighter than its own tracking (p90 lateral error 2.3 cm at 0.25 m), so 18 % of its normal demonstrations contained a stop-and-fix, a knife-edge decision that a 256×144 image cannot resolve.

Commanded wheel speed at pivot	Frames	Truck rotating (|yaw rate| > 0.03 rad/s)
0.00–0.03 m/s	9,704	15 %
0.03–0.06 m/s	6,948	26 %
0.06–0.09 m/s	4,415	45 %
0.09–0.12 m/s	3,072	60 %
0.12–0.20 m/s	4,223	75 %
0.20–1.00 m/s	2,977	88 %

The chain is: unnecessary stops in the demonstrations → the policy cannot tell when to pivot, so it starts pivots it does not need → the final pivot commands taper to a creep that barely moves the truck → stall. Expert v4.1 widened the band, made pivots decisive (at least 0.2 m/s) and removed contradictory "stop" labels from the perturbations (Section 3.1). In validation the new expert docked 100 % of normal starts straight in, without a stop (v4: 82 %), and the policy trained on its data pivots at all in 7 of 128 starts instead of 63.

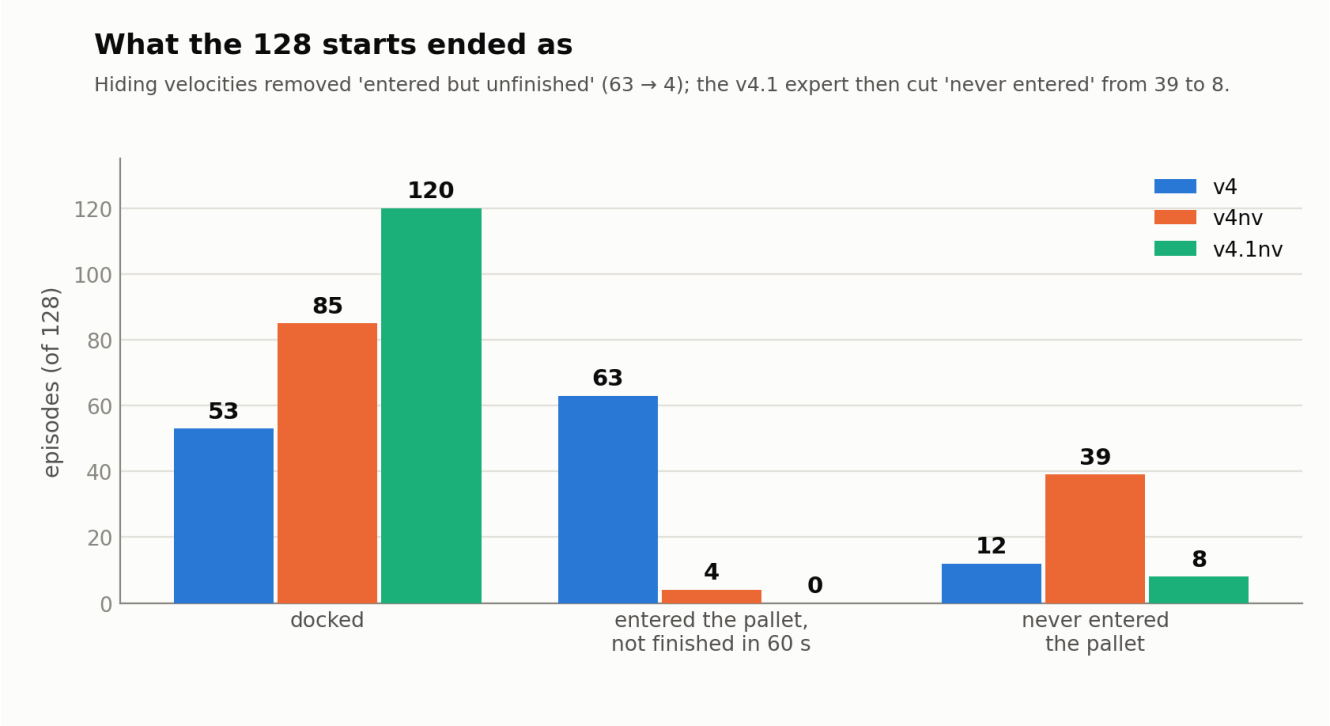
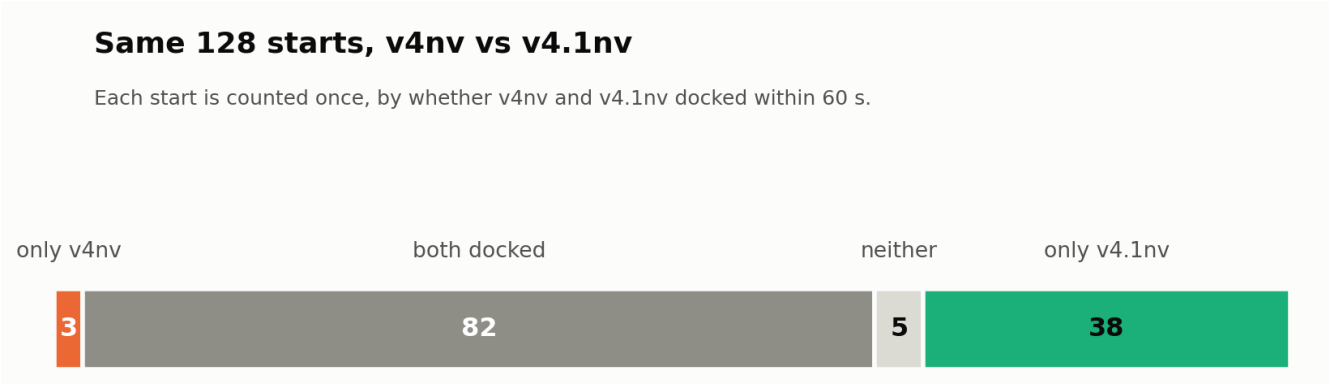


Figure 4. Mutually exclusive outcomes of the 128 starts (v4: 53 + 63 + 12; v4nv: 85 + 4 + 39; v4.1nv: 120 + 0 + 8).



```
p < 0.001 (two-sided sign test on the 41 discordant starts: 3 vs 38)
```

Figure 5. The same 128 starts, v4nv vs v4.1nv: 38 docked only with v4.1nv, 3 only with v4nv.

4.3 Earlier generations and inconclusive results

Generation	Change	Result (first roll)
v1	frozen vision encoder; 962 demos with neighbouring environments visible in the camera	8/32 (25 %)
v2	two switch bits added to the state	17/32 (53 %)
v3	clean data (40 m spacing), expert v3, vision tower unfrozen	30/64 (47 %); about 22 of the failures wedged inside the pocket
v4	expert v4 (full motion set, real speeds), wheel-speed action	23/64 (36 %); all failures were time-outs, no jams
v4d1	DAgger round 1: policy rollouts labelled by a stateless version of the expert	17/64 (27 %); the policy started pivots at the standoff and never finished them

Three lessons from this history. (i) A perception probe showed that neighbouring environments visible in the image cost the frozen features a factor of 2.4 in precision (1.74 → 4.10 cm), so all v1/v2 data were discarded and the spacing raised to 40 m. (ii) The "worse" ranking among v3, v4 and v4d1 is **not significant**: re-rolling the 41 starts v4 had "failed" at 80 s docked 26 of them within 160 s, so a single 64-episode roll of a stochastic policy carries about ±4 episodes of noise per policy. The one robust signal there is v4d1's pivot loop. (iii) The DAgger round tested the right idea (on-policy state coverage) with a weak tool: the stateless labeller docked only 4 of 11 pushed-in recoveries in simulation. Adversarial imitation (GAIL, AIRL) has not been tried; the failures we found are not what it fixes, the expert can be queried at any state, and it would need reinforcement learning in the loop at roughly 60 simulated steps per second.

4.4 Remaining failures

Eight of 128 starts do not dock; all eight never enter the pallet, and they spend a median 36 s at pivot steering: the same stall as before, now in 6 % of starts instead of 30 %. No episode jammed. Nothing here tells us how the policy behaves on starts outside the training distribution.

5. Discussion and limitations

What we can claim. In simulation, on starts drawn from the training distribution, a 450 M-parameter VLA fine-tuned on 704 expert demonstrations docks 94 % of random starts within 60 s with a median time equal to the expert's. The improvement from 41 % to 94 % is measured on identical

starts and is statistically clear at each step.

What we cannot claim.

- *Attribution.* The three v4.1 expert changes were made together, together with 12 k further fine-tuning steps and 24 % fewer training frames, in one training run. We know the combination works and that the stall disappeared, not which change mattered most, and we have no variance across training seeds.
- *Generalisation.* All evaluations use the training start distribution (offsets ±0.3 m, ±15°, one pallet, an empty floor). Larger offsets, other distances and distractors are untested.
- *Reality.* The pallet is a fixed rigid body with a simplified collider; the switch geometry, the steering-sign convention of the 155° side and the pocket tolerance are assumptions; there is no camera noise, exposure change, latency or actuator imperfection beyond the simulator's own physics.
- *Time budget.* The 60 s budget is our choice; the expert's own times are 18 s (median) and 38 s (p95).
- *Known data gap.* In v4.1 the kicks are noticed before the pocket, so the data contain no recoveries from *inside* the pocket; the wiggle-inside-the-pocket skill planned for v5 is not part of this result.

6. From simulation to hardware

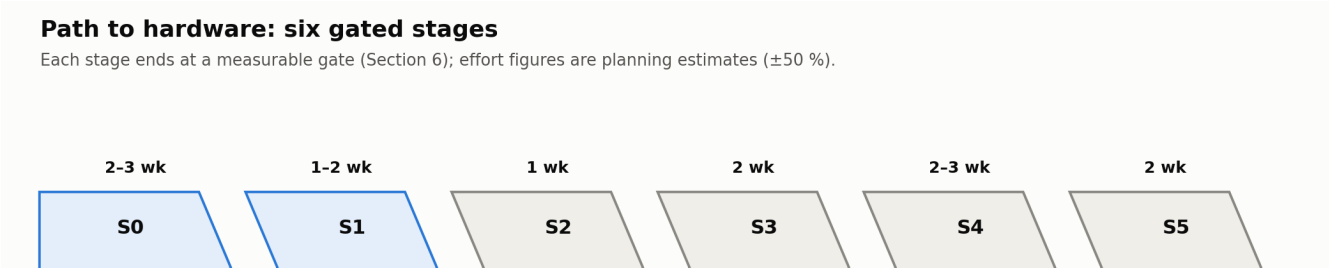
6.1 What transfers

The deployed policy needs only: the **camera image** (the same Femto Bolt mounted on the fork carriage, scaled to 256×144), the **steering angle**, and the **two backrest-switch bits**. Because the velocities are hidden, **no wheel-speed or odometry estimate is needed**, which removes a noisy dependency; depth is not used. It outputs a steering target and a wheel ground-speed target at 20 Hz, which map onto a tricycle drive controller.

6.2 Sim-to-real gap inventory

Gap	In simulation today	On the real truck	Risk	Mitigation
Appearance	one green plastic pallet, empty floor, fixed light	racks, other pallets, people, glare, different pallet colours and materials	high	randomise backgrounds, textures, lighting, exposure and sensor noise; fine-tune on real frames
Camera	ideal pinhole, no delay	auto-exposure, blur, 50–100 ms latency, mounting tolerance of mm and degrees	high	extrinsic and intrinsic jitter, latency injection, calibration, time synchronisation
Compute and latency	the simulator waits for the policy (about 0.2 s per chunk on an RTX 4060 laptop GPU)	a real-time loop on the truck's compute	high	benchmark the target hardware; asynchronous chunk execution; lower-precision inference; re-validate with a larger executed prefix
Actuation	rate limit 0.6 rad/s, instantaneous wheel velocity; weak commands (< 0.09 m/s) barely rotate the truck at 90°	motor-controller dynamics, backlash, stiction, floor friction, load	high	system identification of the dead-zone and slip; dynamics randomisation; keep the decisive-command property of the expert
Turning speed	the simulated truck slides sideways when turning in reverse above about 0.45 m/s	tyre and floor dependent	medium	speed caps at first; measure on the real floor
Pallets and contact	one pallet type, kinematic, simplified collider	wooden, plastic, damaged, different pocket sizes; real friction	medium	randomise pallet type, size and pose within the standards; verify the collider
Switches and geometry	assumed ±0.216 m, 10 mm trigger; steering sign of the 155° side assumed	measured values	medium	measure and update the simulation
Safety	none inside the policy	people, racks, other vehicles	critical	independent safety layer (6.4)

6.3 Staged plan with gates



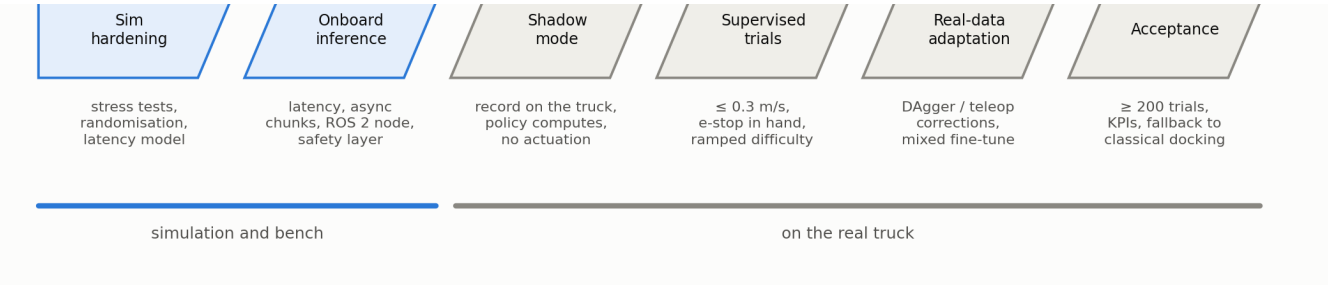


Figure 6. Six gated stages; effort figures are planning estimates (±50 %), about 10–14 weeks in total.

Stage	Work	Gate to proceed (proposed)
S0 Sim hardening (2–3 wk)	Stress evals of the current policy (wide lateral ±0.6 m, wide heading ±30°, near 1.5–2.5 m and far 5–6 m starts); decide between a wiggle-inside-the-pocket skill (v5) and wider start distributions; ablations and a second seed; domain randomisation (appearance, camera, friction, latency, state noise); a latency model in the loop.	≥ 90 % docked and 0 jams on a randomised suite of ≥ 256 starts including injected latency of 0.2–0.5 s; consistent across two seeds.
S1 Onboard inference and integration (1–2 wk)	Measure end-to-end latency on the truck's compute; asynchronous chunk execution (compute the next chunk while executing the current one); bf16 or INT8 and fewer denoising steps if needed, each re-validated in simulation; ROS 2 node (camera, steering encoder, switches in; steering and speed out); the safety layer.	The policy delivers a fresh chunk before the executed prefix runs out; replaying logged simulation episodes through the real software stack reproduces the simulated actions.
S2 Shadow mode (1 wk)	A human drives; the policy computes actions and nothing is actuated. Record images, state and policy outputs across lighting and pallet variety.	Policy actions agree with the human in the approach phase; an image-statistics check shows no gross domain gap; no crashes, NaNs or timeouts.
S3 Supervised low-speed trials (2 wk)	Speed capped at 0.2–0.3 m/s, fork lowered, e-stop in hand, safety layer active; easy poses first, difficulty ramped.	20 consecutive clean docks from easy offsets (zero failures in 30 trials bounds the failure rate below about 10 % at 95 % confidence); then ≥ 50 trials over the intended distribution with ≥ 90 % success and no contact incidents; every abort explained.
S4 Real-data adaptation (2–3 wk)	Operator corrections (human-gated DAgger) and, if useful, the existing pose-estimation pipeline as an automatic labeller; fine-tune on a sim and real mixture.	Real success rate improves over S3 on held-out real trials.
S5 Acceptance (2 wk)	≥ 200 trials over lighting, pallet types and offsets; a defined fallback (retry, hand-over to an operator, or the classical dock).	Success within the budget ≥ 95 % (lower interval bound ≥ 90 %), fewer than one contact incident per 200, median ≤ 25 s.

6.4 Safety architecture

The learned policy is never the only safeguard: **the policy proposes, an independent supervisor disposes.** The supervisor runs separately from the policy and enforces: steering and rate limits and speed caps by zone; a keep-out bubble from the truck's two 2D lidars (people and obstacles); watchdogs on camera-frame age, inference age and invalid actions, each causing a controlled stop; **wedge detection from the traction-motor current against the commanded speed** (exactly the signal hidden from the policy); an interlock on the backrest switches; and a hardware emergency-stop chain. Every run logs images, states, actions and supervisor interventions.

6.5 Hardware metrics and statistics

Report, per condition: success within the budget with a Wilson interval, median and p90 docking time, contacts and jams per 100 docks, supervisor interventions per 100 docks, and aborts by cause. Compare conditions on identical starts where possible (paired tests), and keep the evaluation protocol of Section 3.4.

6.6 Decisions needed from the project owner

1. What compute is available on the truck, and is an off-board GPU over a network acceptable? This sets S1's latency budget.
2. Which pallet types, colours and environments must be supported at first?
3. Acceptance targets: is a 60 s budget and 95 % success right, and what contact limit is allowed?
4. The steering-sign convention of the 155° side, the real switch positions and their trigger distance (to update the simulation).
5. When is the in-pocket wiggle skill (v5) needed: only if stress tests or real trials show wedges after entry.
6. Is the existing pose-estimation pipeline available as a labeller for real frames in S4?

7. Conclusions

A small VLA policy, trained by imitation of a privileged scripted expert in simulation, docks a pallet truck from one camera, the steering angle and two switch bits in 94 % of random starts within 60 s, matching the expert's median time. The path from 41 % to 94 % came from understanding what the policy did in closed loop and why the demonstrations taught it: removing a velocity shortcut and removing unlearnable, weakly actuated stop-and-fix decisions from the expert. The result is a simulation result from single runs; the credible route to hardware is the gated plan above, whose first stage (stress tests, randomisation and a latency model) can start immediately with the existing policy.

References

1. M. Shukor et al., "SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics," arXiv:2506.01844, 2025.
2. A. Marafioti et al., "SmolVLM: Redefining small and efficient multimodal models," arXiv:2504.05299, 2025.
3. Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le, "Flow Matching for Generative Modeling," ICLR 2023 (arXiv:2210.02747).
4. K. Black et al., "π0: A Vision-Language-Action Flow Model for General Robot Control," arXiv:2410.24164, 2024.
5. P. de Haan, D. Jayaraman, S. Levine, "Causal Confusion in Imitation Learning," NeurIPS 2019.
6. S. Ross, G. Gordon, D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," AISTATS 2011 (DAgger).
7. M. Laskey et al., "DART: Noise Injection for Robust Imitation Learning," CoRL 2017.
8. J. Ho, S. Ermon, "Generative Adversarial Imitation Learning," NeurIPS 2016.
9. J. Tobin et al., "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," IROS 2017.
10. M. Mittal et al., "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments," IEEE RA-L 2023 (Isaac Lab's predecessor).

Citation details should be checked before external submission.

Appendix A. Reproducibility

Everything lives under `/home/abhinav/Data_Drive/agile_amr` ; the full engineering log, tables and file map are in `README.md` , Section 10. Main entry points:

```
python expert_dock.py --num_envs 1 --episodes 4 --device cpu --viz kit                  # watch th
python expert_dock.py --num_envs 32 --episodes 2 --noise_frac 0.5 --push_frac 0.15 --headless --ou
python to_lerobot.py "<dir>/demos.h5" <dataset> && python make_nv_dataset.py <dataset> <dataset_nv
python train_vision.py --policy.path=<checkpoint> --dataset.root=<dataset_nv> --batch_size=8 --ste
bash view_failures.sh v41nv --fresh                                                       # watch
bash pipeline_v41.sh                                                                      # collec
python make_figures.py && python make_diagrams.py                                         # regene
```

Artifacts: `outputs/smolvla_dock_v41nv` (current best policy), `data/exp_v41` (demonstrations and 128-start evaluation), `data/lerobot/dock_v41nv` (dataset). The v3 and v4 checkpoints and their evaluations are kept read-only.

Appendix B. Hyperparameters of the current policy (v4.1nv)

Setting	Value
Base	v4nv checkpoint (chain: SmolVLA base → v1 → v2 → v3 → v4 → v4nv → v4.1nv)
Optimiser	AdamW, β = (0.9, 0.95), weight decay 1e-10, gradient clip 10
Learning rate	2e-5 peak, 200 warm-up steps, cosine decay to 2.5e-6 over 12,000 steps
Batch / steps / time	8 / 12,000 / about 1.8 h on one RTX 4060 (8 GB)
Trainable parameters	about 198 M of 450 M (vision encoder, connector, action expert, projections)
Chunk / executed / denoising steps	50 / 3 / 10
State normalisation	mean/std from the dataset; velocity columns zeroed
Augmentation	none
Final training loss	0.006 (flow-matching MSE; it did not predict closed-loop success in this study)